

คู่มือการใช้งาน GitHub แบบ Step by Step

เอกสารนี้อธิบายการใช้งาน GitHub แบบเป็นขั้นตอน เหมาะสำหรับผู้เริ่มต้นที่ต้องการทำงานกับ repository, push/pull, branch และ merge

สิ่งที่ควรรู้ก่อนเริ่ม

Git กับ GitHub ต่างกันอย่างไร

- Git คือระบบจัดการเวอร์ชันของโค้ด
- GitHub คือเว็บไซต์สำหรับเก็บ repository และทำงานร่วมกัน

สิ่งที่ต้องมี

1. สมัครบัญชี GitHub ที่ <https://github.com/>
2. ติดตั้ง Git จาก <https://git-scm.com/>
3. มีโพลเดอ์โปรเจกต์บนเครื่อง
4. เปิด Terminal เช่น PowerShell, Command Prompt หรือ Terminal ใน VS Code/Cursor

เช็กว่า Git พร้อมใช้งาน

```
git --version
```

ถ้าระบบแสดงเวอร์ชัน เช่น git version 2.x.x แปลว่าใช้งานได้

1. การสร้าง Repository

การสร้าง repository ทำได้ 2 แบบ

- สร้างบน GitHub ก่อน แล้วค่อยเชื่อมกับโปรเจกต์ในเครื่อง
- สร้างจากโปรเจกต์ในเครื่อง แล้วค่อยส่งขึ้น GitHub

วิธีที่ 1: สร้าง Repository บน GitHub ก่อน

1. เข้าเว็บไซต์ GitHub และล็อกอิน
2. กดปุ่ม New repository
3. ตั้งชื่อ repository
4. เลือกว่าจะเป็น Public หรือ Private
5. กด Create repository

หลังสร้างเสร็จ GitHub จะให้ URL ของ repository เช่น

```
https://github.com/your-username/your-repo.git
```

วิธีที่ 2: สร้างโปรเจกต์ในเครื่องก่อน

1. เปิด Terminal ไปยังโฟลเดอร์โปรเจกต์
2. รันคำสั่งด้านล่าง

```
git init
```

คำสั่งนี้จะสร้าง repository ของ Git ในโฟลเดอร์ปัจจุบัน

เพิ่มไฟล์เข้า staging และ commit ครั้งแรก

```
git add .  
git commit -m "Initial commit"
```

ความหมายของคำสั่ง

- `git add .` เพิ่มไฟล์ทั้งหมดเข้า staging area
- `git commit -m "Initial commit"` บันทึกการเปลี่ยนแปลงเป็น commit

เชื่อมโปรเจกต์ในเครื่องกับ GitHub

หลังจากสร้าง repository บน GitHub แล้ว ให้เชื่อมด้วยคำสั่ง

```
git remote add origin https://github.com/your-username/your-repo.git
```

เช็คว่าเชื่อมสำเร็จหรือยัง

```
git remote -v
```

2. การ Push และ Pull

push คืออะไร

push คือการส่ง commit จากเครื่องเราไปยัง GitHub

pull คืออะไร

pull คือการดึงการเปลี่ยนแปลงล่าสุดจาก GitHub ลงมาในเครื่อง

Push ครั้งแรก

หลัง commit แล้ว ให้ส่งขึ้น GitHub ด้วยคำสั่ง

```
git branch -M main
git push -u origin main
```

ความหมายของคำสั่ง

- git branch -M main เปลี่ยนชื่อ branch ปัจจุบันเป็น main
- git push -u origin main ส่ง branch main ไปยัง remote ชื่อ origin
- -u ทำให้ครั้งต่อไปใช้ git push หรือ git pull ได้เลย

ขั้นตอนการทำงานปกติหลังแก้ไฟล์

1. แก้ไฟล์ในโปรเจกต์
2. เช็คสถานะ

```
git status
```

3. เพิ่มไฟล์เข้า staging

```
git add .
```

4. commit

```
git commit -m "อธิบายสิ่งที่แก้"
```

5. push ขึ้น GitHub

```
git push
```

ดึงงานล่าสุดจาก GitHub

```
git pull
```

ถ้าต้องการดึงจาก branch ที่ระบุชัดเจน

```
git pull origin main
```

กรณีเริ่มงานบนเครื่องใหม่

ถ้ามี repository อยู่บน GitHub แล้ว และต้องการดึงลงเครื่อง ให้ใช้คำสั่ง

```
git clone https://github.com/your-username/your-repo.git
```

3. การสร้าง Branch

Branch คืออะไร

branch คือเส้นการพัฒนาแยกออกจากงานหลัก เพื่อให้แก้ feature หรือ fix bug ได้โดยไม่กระทบ branch หลัก

ดู branch ที่มี

```
git branch
```

สร้าง branch ใหม่

```
git branch feature-login
```

สลับไป branch ใหม่

```
git checkout feature-login
```

สร้างและสลับ branch ในคำสั่งเดียว

```
git checkout -b feature-login
```

สำหรับ Git เวอร์ชันใหม่

```
git switch -c feature-login
```

ส่ง branch ใหม่ขึ้น GitHub

```
git push -u origin feature-login
```

ตัวอย่าง workflow ของ branch

1. อยู่บน main
 2. สร้าง branch ใหม่ เช่น feature-login
 3. แก้โค้ดใน branch นี้
 4. add, commit, push
 5. เปิด Pull Request บน GitHub
 6. merge กลับเข้า main
-

4. การ Merge Branch

merge คืออะไร

merge คือการนำการเปลี่ยนแปลงจาก branch หนึ่งไปรวมกับอีก branch หนึ่ง

วิธี merge ผ่านเครื่อง

สมมติว่าเราต้องการ merge feature-login เข้า main

1. สลับกลับไป main

```
git checkout main
```

2. ดึงข้อมูลล่าสุดก่อน

```
git pull origin main
```

3. merge branch ที่ต้องการ

```
git merge feature-login
```

4. push ขึ้น GitHub

```
git push origin main
```

วิธี merge ผ่าน GitHub

1. push branch งานขึ้น GitHub
2. เข้า repository บน GitHub
3. กด Compare & pull request
4. ใส่ชื่อและรายละเอียดของงาน
5. กด Create pull request
6. ตรวจสอบความถูกต้อง

7. กด Merge pull request

8. กด Confirm merge

ลบ branch หลัง merge

ลบในเครื่อง

```
git branch -d feature-login
```

ลบบน GitHub

```
git push origin --delete feature-login
```

5. การแก้ปัญหา Merge Conflict

บางครั้ง merge ไม่สำเร็จทันที เพราะมีการแก้ไขไฟล์เดียวกันในหลาย branch

ตัวอย่างอาการ

เมื่อรัน git merge หรือ git pull อาจเจอ conflict

วิธีแก้

1. เปิดไฟล์ที่มีปัญหา
2. จะเห็นสัญลักษณ์ประมาณนี้

```
<<<<<< HEAD
โค้ดของ branch ปัจจุบัน
=====
โค้ดของ branch ที่จะ merge เข้า
>>>>>> feature-login
```

3. เลือกว่าจะเก็บโค้ดส่วนไหน หรือรวมทั้งสองส่วน
4. ลบสัญลักษณ์ conflict ออก
5. บันทึกไฟล์
6. เพิ่มไฟล์กลับเข้า staging

```
git add .
```

7. commit ผลการแก้ conflict

```
git commit -m "Resolve merge conflict"
```

6. คำสั่งพื้นฐานที่ใช้บ่อย

```
git status
git add .
git commit -m "message"
git push
git pull
git branch
git checkout branch-name
git checkout -b new-branch
git merge branch-name
git clone <repo-url>
```

ความหมายแบบสั้น

- `git status` ดูสถานะไฟล์
- `git add .` เตรียมไฟล์ก่อน commit
- `git commit -m "message"` บันทึกการเปลี่ยนแปลง
- `git push` ส่งงานขึ้น GitHub
- `git pull` ดึงงานล่าสุดลงเครื่อง
- `git branch` ดูรายชื่อ branch
- `git checkout` สลับ branch
- `git merge` รวม branch
- `git clone` คัดลอก repository จาก GitHub ลงเครื่อง

7. สิ่งที่ต้องทำในการทำงานจริง

1. pull ก่อนเริ่มงาน ถ้าทำงานร่วมกับคนอื่น
2. แยก branch ตามงาน เช่น feature/, fix/, docs/
3. commit ให้สั้น ชัด และสื่อความหมาย
4. อย่าแก็งงานหลายเรื่องใน commit เดียว
5. push งานขึ้น GitHub สม่าเสมอ
6. ใช้ Pull Request ก่อน merge งานสำคัญ

ตัวอย่างชื่อ commit ที่ดี

```
add login page
fix navbar alignment
update README instructions
```

8. ตัวอย่างลำดับการใช้งานตั้งแต่ต้นจนจบ

กรณีเริ่มโปรเจกต์ใหม่

```
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/your-username/your-repo.git
git push -u origin main
```

กรณีทำ feature ใหม่

```
git pull
git checkout -b feature-login
# แก้ไฟล์ในโปรเจกต์
git add .
git commit -m "add login feature"
git push -u origin feature-login
```

จากนั้นให้เปิด Pull Request บน GitHub แล้ว merge เข้า main

กรณีอัปเดตงานจากทีม

```
git checkout main
git pull origin main
```

9. อื่นๆ ที่ควรรู้

README.md

ใช้สำหรับอธิบายโปรเจกต์ เช่น วิธีติดตั้ง วิธีใช้งาน และข้อมูลสำคัญของ repository

.gitignore

ใช้ระบุไฟล์หรือโฟลเดอร์ที่ไม่ต้องการให้ Git ติดตาม เช่น

```
node_modules/  
.env  
dist/
```

Public กับ Private

- Public คนอื่นมองเห็นได้
- Private มองเห็นได้เฉพาะคนที่ได้รับสิทธิ์

Pull Request คืออะไร

Pull Request หรือ PR คือคำขอรวมโค้ดจาก branch หนึ่งไปอีก branch หนึ่ง พร้อมให้คนอื่น review ก่อน merge

10. สรุป

การใช้งาน GitHub พื้นฐานมีลำดับสำคัญดังนี้

1. สร้าง repository
2. เชื่อมโปรเจกต์ในเครื่องกับ GitHub
3. add และ commit งาน
4. push ขึ้น GitHub
5. ใช้ branch แยกงาน
6. merge งานกลับ branch หลัก
7. แก้ conflict เมื่อจำเป็น

ถ้าฝึกทำตามขั้นตอนเหล่านี้บ่อย ๆ จะใช้งาน GitHub ได้คล่องขึ้น และทำงานร่วมกับผู้อื่นได้ง่ายขึ้น